

Senior Project CENG 4265

Self Driving RC
Project Proposal

Ethan Wells
Alexis Perez
Ethan Bishop
Abigail Duran

University of Houston - Clear Lake
September 1st, 2026

Project Proposal

Executive Summary:

Self-Driving Car systems require real time and low- latency processing. In this project, we will be recreating this design to a more compact design. The primary objective of this senior design project is to gain hands-on expertise in custom printed circuit board (PCB) hardware design and machine learning model development.

The methodology for this project is to build a custom circuit board around a small computer module such as a RaspberryPi. And uploading a trained vision model, that was trained on Public Traffic Sign datasets. The Self-Driving RC car will take around one year to complete. Phase one focuses on designing the stationary system by creating a custom PCB board and training a visual model. Once built, testing for latency and accuracy. Phase two would focus on implementing the stationary system onto a custom-built vehicle that would simulate real-world self-driving vehicles.

Introduction:

Self-Driving vehicles use a real-time sign recognition system. Our system faces the challenge of processing video feeds with minimal latency while operating under strict processing capabilities and physical space constraints. Compared to real-world self-driving vehicles relying on large resource-heavy platforms, smaller applications require more compact and efficient solutions.

This project addresses that challenge by designing a compact Self-Driving RC Car from the ground up. During the fall semester we will be designing a custom PCB around a small compute module such as a Raspberry Pi and pairing it with an optimized learning model. The completed stationary device will capture images through a camera, process them locally, and identify traffic signs without relying on cloud-based processing. For the Spring Semester, the focus will shift from a stationary device to a vehicle implementation. The custom PCB and vision model will be mounted onto a custom-built vehicle. The combined system would demonstrate the real-time visual sign recognition of self-driving vehicles. Ultimately, this project serves as a practical implementation, by bridging embedded hardware design with machine learning integration.

Project Statement:

The goal of this project is to build a self-driving RC car that drives itself through a controlled course while recognizing and responding to traffic signs along the way. A camera looks at each sign, the car figures out what it is, and it acts on it — stopping at a stop sign, slowing down for

a speed limit — all while keeping itself on the course. Traffic sign recognition is the heart of the project, it's where we started and what everything else is built around, but the finished product is a small autonomous vehicle, not just a device that reads signs. We're building it around a Raspberry Pi Compute Module mounted on a circuit board we design ourselves.

The point of the project isn't to do something that's never been done — plenty of people have built traffic sign recognition, and plenty of people have built self-driving RC cars. The point is for our team to learn a stack of things we haven't done before: designing a PCB, training and deploying a machine learning model on embedded hardware, and getting a vehicle to sense its surroundings and drive itself. It's also enough work to keep four of us busy across two semesters, which is what we set out to scope.

We've split the work into two phases, one per semester, and each phase ends in a working demo.

In the first phase (fall) we build the recognition system as a stationary unit. We train a vision model to tell apart a set of common U.S. road signs — around 10 to 15 of them like stop, yield, and speed limits — using an existing dataset (the LISA Traffic Sign Dataset), and we get it running directly on the Compute Module through TensorFlow Lite. We pair it with a camera and our custom carrier board, so that by the end of the semester we have a self-contained device that captures an image, recognizes the sign, and shows what it sees, all on its own hardware with no laptop attached. We'll count this phase a success when the device correctly identifies the signs it was trained on at least 90% of the time, and does it fast enough to feel real-time — a few frames per second — running on the Pi itself.

In the second phase (spring) we mount that system on an RC car and turn it into a self-driving vehicle. We add the chassis, motors, a motor controller, and steering, plus the sensors the car needs to know where it is: ultrasonic sensors to keep it off the walls and catch obstacles, and an IMU to keep it driving straight and turning cleanly. The camera stays the primary sensor, since reading signs is a visual job and nothing else can do it. We then grow the software from just recognizing signs into making driving decisions — taking the camera and the sensor readings and turning them into steering and motor commands. We'll count this phase a success when the car can drive a controlled course from start to finish on its own, staying on the path and correctly responding to the signs it passes, at a safe low speed.

Deliverables:

Phase 1 — Stationary Recognition System (Fall)

1. Trained model — A model trained in Python to recognize our set of U.S. signs, using an existing dataset (the LISA Traffic Sign Dataset). Once it's trained we convert it to TensorFlow Lite so it can run on the Pi.

2. Camera and image pipeline — Code using OpenCV to pull frames off the camera, find the sign in the frame, and hand it to the model.
3. On-device recognition — The model running directly on the Compute Module through TensorFlow Lite, classifying signs without a laptop attached.
4. Custom PCB — A carrier board we design ourselves that the Compute Module sits on, tying the camera, LCD, power, and (later) the vehicle electronics together on one board.
5. LCD output — The screen that prints out which sign the device just recognized.
6. Working stationary unit — Everything put together into one self-contained device for the stationary demo, recognizing signs held in front of the camera.

Phase 2 — Autonomous Vehicle (Spring)

7. RC vehicle platform — A chassis with motors, a motor controller, and steering, running off its own onboard power and carrying the Compute Module and our board.
8. Sensor integration — Ultrasonic sensors and an IMU tied into the system through our PCB, giving the car obstacle distance and heading on top of the camera.
9. Vehicle control software — The code that turns decisions into movement: motor control, steering logic, and an emergency stop.
10. Autonomous decision model — The recognition model grown into a driving model that takes the camera and sensor inputs and decides how the car should move, including how it responds to each sign it recognizes.
11. Integrated self-driving car — The full vehicle, with the camera feed and sensors driving its decisions in real time.
12. Controlled-course demo — The finished car driving a defined course from start to finish on its own, obeying the signs along the way.

Stretch goal: speaker output — if the core system works, we'd like the car to announce out loud which sign it just recognized.

Methodology:

Two phases, one per semester. We're building the hard part first. The whole project depends on the car being able to see and understand a sign, so we spend the fall getting that right as a stationary unit — model, camera, and custom board on the bench — before we ever put wheels under it. That way, when we start the vehicle in the spring, the perception system is already proven and we're adding driving on top of something that works, instead of debugging everything at once. The disadvantage is that the fall demo is "just" a device on a table, which looks less impressive than a moving car, but it's the safer order and it's what keeps the spring semester from turning into a scramble.

Parallel hardware and software tracks. Within each phase we split into two tracks that run at the same time: the software, meaning training and running the model and later the driving

logic, and the hardware, meaning the PCB and wiring everything together. We do it this way because the PCB has a long lead time — it can take weeks to get a board back once it's ordered — so we don't want the software sitting idle waiting on it. Early on we bring everything up on the Raspberry Pi Compute Module I/O Board — the official development board that breaks out the module's interfaces — to prove the camera-to-LCD path works, and we design our custom carrier board in parallel. Once our board comes in, we swap it in.

Vehicle and sensors (spring). In the spring we build the car around the same Compute Module and board. The camera stays the primary sensor because sign recognition is a visual task; the ultrasonic sensors and IMU are there for the driving itself — distance to walls and obstacles, and heading for straight lines and clean turns. The ultrasonic sensors are timing-sensitive, and the Pi runs Linux, which isn't real-time, so if the readings come back too jittery we can put a small microcontroller (an RP2040) on our own board to handle the sensor timing and report clean numbers back to the Pi. Either way it stays on one board — no separate Arduino. We deliberately left LiDAR out: it can't read a sign, and the mapping and software stack it needs is more than a defined start-to-finish course calls for.

From recognition to decisions. The model starts as a classifier that names the sign in front of it. In the spring we grow it and the software around it into a decision layer that takes the sign the camera sees, plus the distance and heading from the sensors, and turns all of it into motor and steering commands — go, stop, slow down, steer to stay on the course. An emergency stop sits over the top of it so we can always kill the motors.

Testing. We test the same way in both phases: prove each piece on its own, then together. In the fall that means checking the model's accuracy and speed on the Pi and running the full stationary unit against signs held in front of the camera. In the spring it means bench-testing the motors, steering, and sensors first, then low-speed runs on a controlled course, tightening the model and the control logic between runs until the car can finish the course on its own.

Things to Learn:

- Machine learning — training a model to classify images, and getting a dataset ready to train on.
- Getting a model onto a Pi — converting it to TensorFlow Lite so it's small and fast enough to run on the Compute Module.
- PCB design — drawing up the schematic and board layout and getting it fabricated.
- OpenCV — capturing camera frames and cleaning them up before they reach the model.
- Raspberry Pi / embedded — running everything on Raspberry Pi OS and getting the camera, LCD, and board working together.
- Vehicle control — driving motors and steering from code, and writing the logic that turns decisions into movement.

- Sensors — reading ultrasonic sensors and an IMU over I²C, and using them to keep the car on the course (and, if we need it, a bit of RP2040 microcontroller firmware).
- Git — keeping our code organized so the whole team can work on it at once.

Resource Requirements:

This is a draft of resources required for the sign-recognition model, custom PCB, and autonomous car platform. The progress of the project and part availability will determine the exact items used.

Software

- Python (model training)
- Raspberry Pi OS (operating system)
- OpenCV (image capture to feed the model)
- TensorFlow Lite (running the model)

Hardware

- Raspberry pi compute module 5 4gb ram
- Raspberry pi compute module 5 I/O board (serves as our bread board)
- Custom PCB I/O board (final design I/O board)
- SD card 32 GB
- 12MP camera
- Speaker
- LCD screen
- Robot car chassis kit (a kit containing the frame, wheels and mounting hardware)
- DC motors
- Motor driver board
- Battery pack
- Voltage regulator

Sensors

- Inertial Measurement Unit (IMU)
- Ultrasonic sensors

Custom PCB

- PCB fabrication
- Connectors, headers, and passive components (resistors, capacitors) for the board

Connectors and Prototyping

- Wiring, and prototyping supplies
- Fasteners, standoffs, and cable management hardware

Enclosure

- 3D printed chassis/enclosure components
- Filament for prototyping and final print

Data Acquisition

- The main objective of data acquisition is to collect and label images of road/traffic signs to train and validate the recognition model

University Resources

- Lab space to complete objectives
- Faculty expertise and guidance
- 3D printing access
- Electricity and internet access

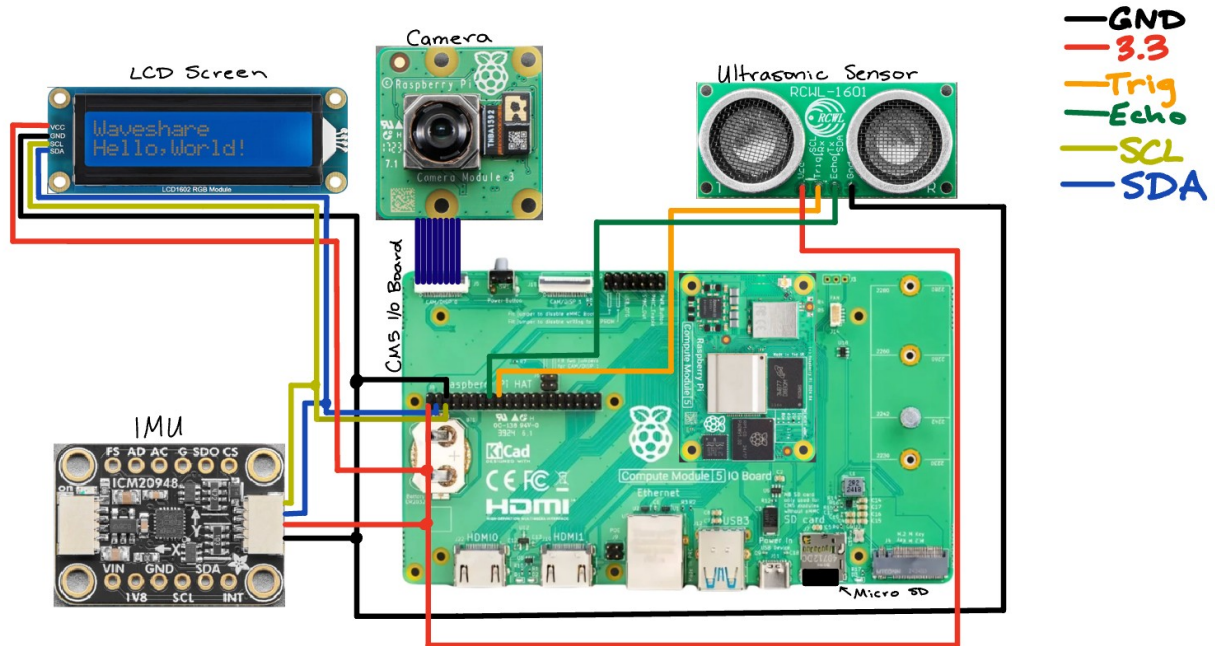
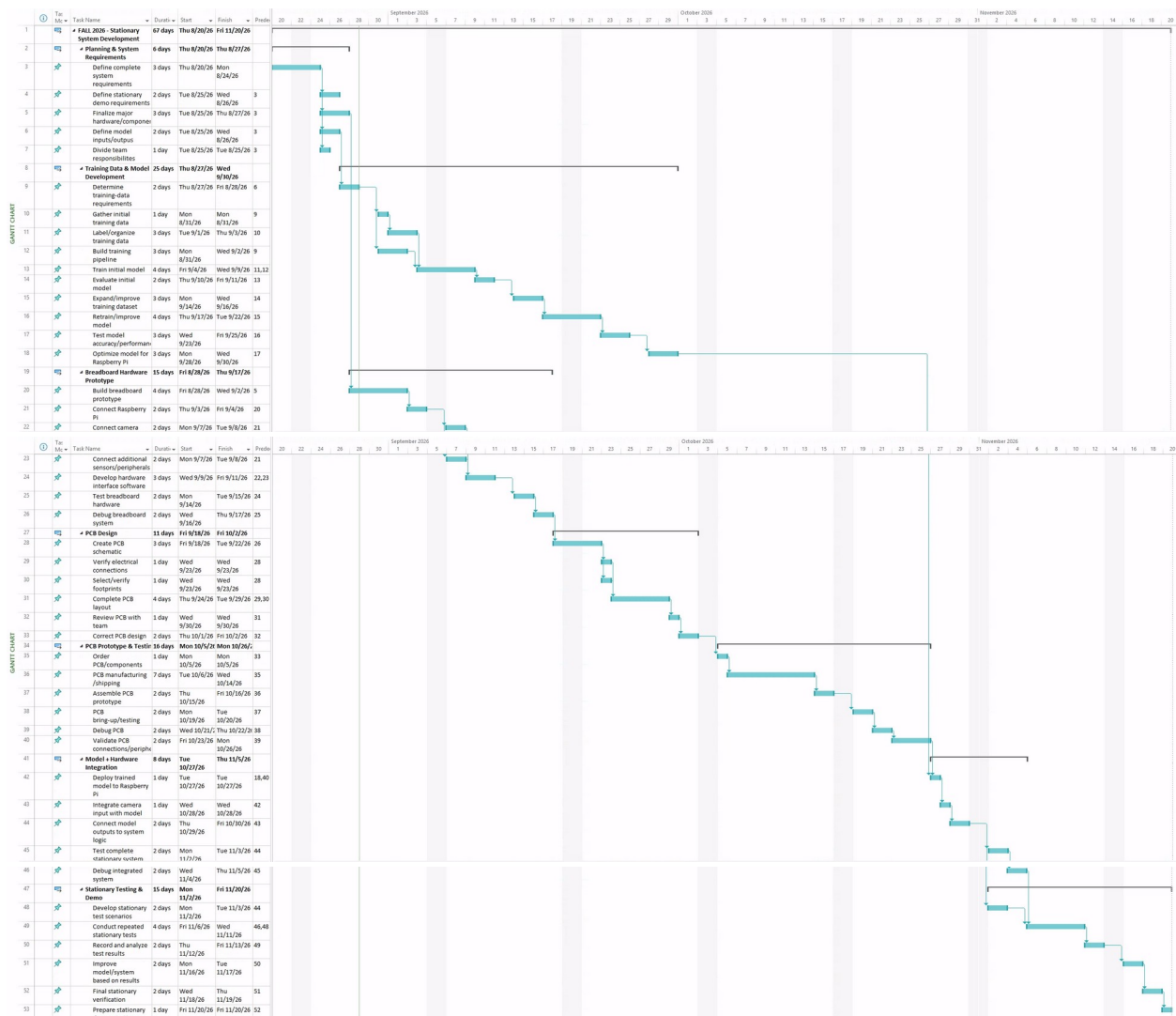


Figure: System Schematic

Schedule: Tentative

Fall 2026 Tentative Schedule



August 20: Begin project planning, define requirements and divide team responsibilities

September 4: Begin model training and breadboard development

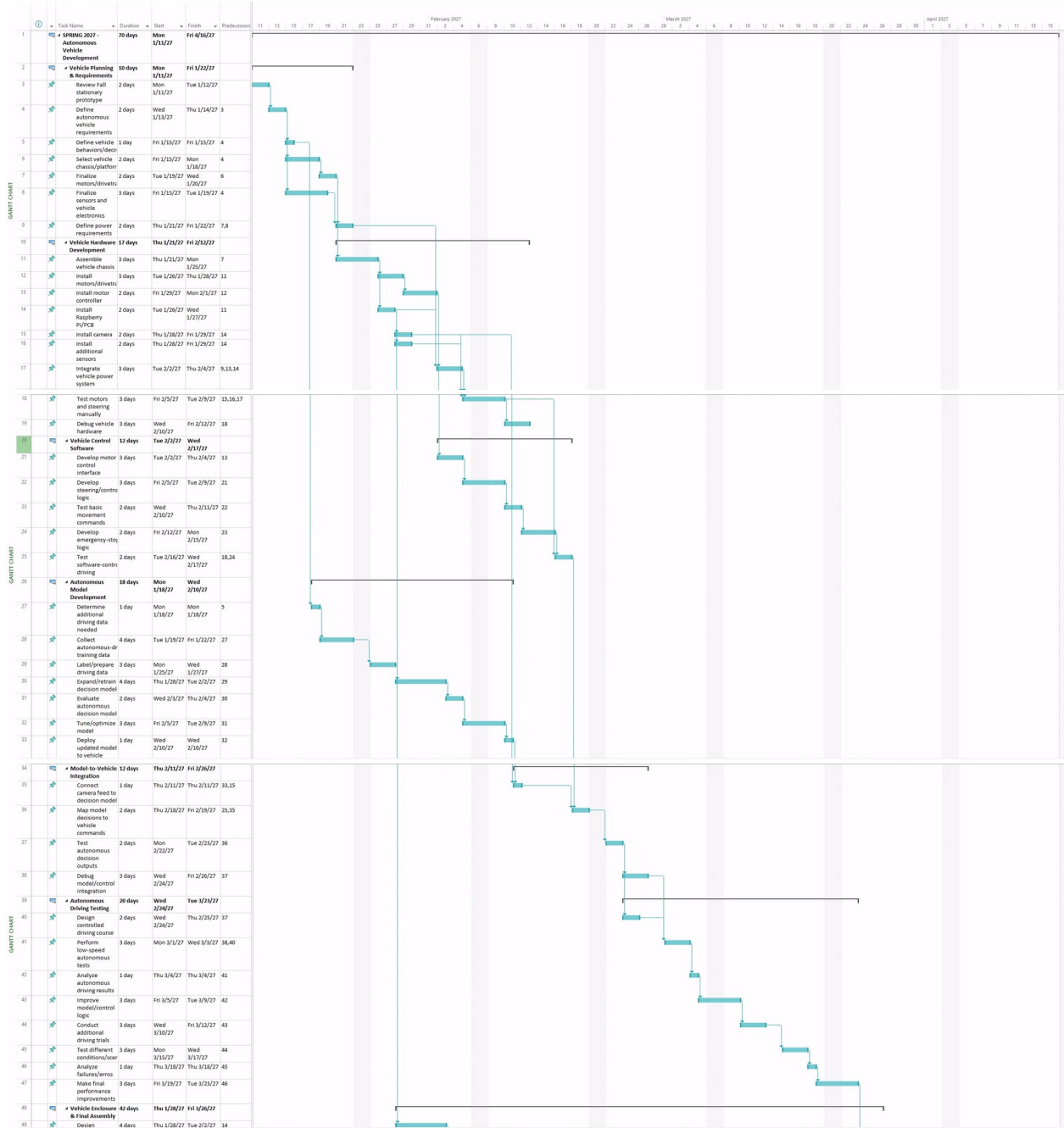
September 30: Complete breadboard prototype and model testing

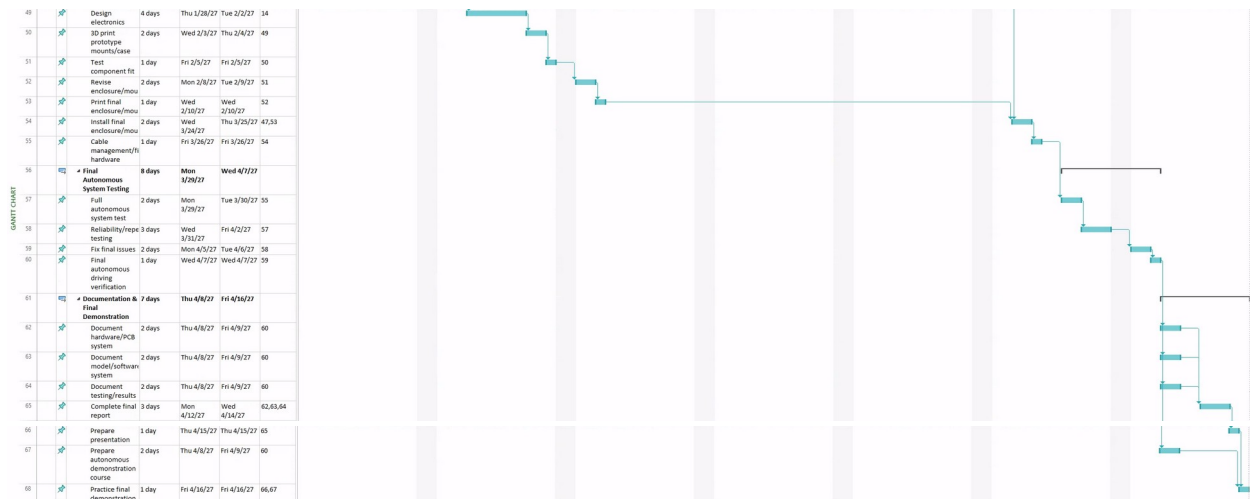
October 23: Complete PCB design and order PCB/components

November 11: Assemble, test, and debug PCB prototype

November 20: Complete model integration and stationary system demonstrations

Spring 2027 Tentative Schedule





January 11: Begin autonomous vehicle planning and hardware selection

January 29: Assemble vehicle chassis and install major hardware

February 19: Complete motor/steering control software and hardware testing

March 12: Retrain/deploy autonomous model and integrate with vehicle controls

March 26: Begin autonomous driving tests and system improvements

April 7: Complete enclosure, final assembly, and hardware cleanup

April 16: Complete final autonomous testing, presentation, and demonstration

Budget: Subject to change

Most of the cost of this project is hardware (compute, camera, chassis, motors, PCB, and sensors).

The following budget breakdown does not include utilities provided by the university or personal items contributed to the project.

Total out of pocket spending:

- ~\$400-500 to be spent at the project's onset (Raspberry Pi, camera, chassis, motors, PCB fabrication, sensors, power system).
- ~\$50-75 per month for unanticipated purchases (replacement parts, additional sensors, enclosure revisions)

Total Budget: \$500 - \$1000 (over two semesters)

Covered by University

- Lab/workplace to build and test
- Electricity and internet access
- 3D printing access

Covered by Team

- Core hardware
- Custom PCB fabrication and components
- Unanticipated purchases and replacement parts

Contacts:

Ethan Wells

Phone: (832) 785-5950

Email: wellse8292@uhcl.edu

Alexis Perez

Phone: (832) 814-5522

Email: pereza4046@uhcl.edu

Abigail Duran

Phone: (832) 493-0256

Email: durana3946@uhcl.edu

Ethan Bishop

Phone: (281) 797-4547

Email: Bishope8630@uhcl.edu